Tutorial Questions | Week 8

COSC2779 - Deep Learning

This tutorial is aimed at reviewing the practical methodology of applying Transformers to natural language processing. Please try the questions before you join the session. The question below has a similar structure to the case study questions that you can expect in the end semester test.

1. You have been hired by a company that provides customer-experience analytics to a large Australian telecommunications provider. Every month the provider receives millions of free-text support messages from customers, and the company wants to process them automatically.

   They have two immediate needs:

   - **Intent routing**: classify each message with a single label (e.g. *billing*, *technical-fault*, *cancellation*) so it can be sent to the right team.
   - **Entity highlighting**: mark every mention of a product, a location, or an account identifier inside the message (a label for every word).

   They have collected 200,000 messages, each annotated with an intent label and with word-level entity tags. Messages range from a few words to several hundred. You decide to build the system with Transformers.

   (a) The model needs numbers, not raw text. Explain how you would turn a message into the sequence of inputs a Transformer consumes. Why not simply split on whitespace and one-hot encode each word?

   Solution:

   - **Tokenise** into sub-word units (e.g. Byte-Pair Encoding / WordPiece). A fixed, modest vocabulary; rare or unseen words are broken into known pieces, so there is no out-of-vocabulary problem (important for product names, typos).
   - Map token ids to dense, learned **embeddings** – not one-hot. One-hot is huge, sparse, and makes every word equally dissimilar; embeddings are low-dimensional and capture similarity.
   - Add special tokens: a `[CLS]` slot at the front (a summary of the whole message) and a `[SEP]` at the end.

   (b) You choose a Transformer over a recurrent network (RNN/LSTM). Give 2 reasons.

   Solution:

   - **Parallelism**: all positions are processed at once (no step-by-step recurrence), so training is far faster on long messages.
   - **Long-range dependencies**: self-attention connects any two tokens directly (path length $\mathcal{O}(1)$ vs $\mathcal{O}(n)$ for an RNN), so distant context is captured without the vanishing-gradient problem.

   (c) Self-attention is built from Queries, Keys and Values. Briefly say what each is, write the scaled dot-product attention, and explain why we divide by $\sqrt{d_k}$.

   Solution: Each token is projected by learned matrices into a **Query** (what it is looking for), a **Key** (what it offers) and a **Value** (its content):

   $$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

   Query–key dot products have variance that grows with d_k; without scaling they become large, the softmax **saturates**, and gradients vanish. Dividing by $\sqrt{d_k}$ keeps them $\mathcal{O}(1)$ so training stays stable.

   (d) Why use multi-head attention instead of a single attention?

Solution: A single attention produces one weighting per token. Multiple heads let the model attend to **several relationships at once** (e.g. syntax, coreference, local vs long-range). Each head has its own W_Q, W_K, W_V with $d_k = d/h$; the heads are concatenated and mixed by W_O, so the cost is about the same as one full-width attention.

(e) Self-attention is a set operation – permute the input tokens and the outputs just permute too. Why is that a problem here, and how is it fixed?

Solution: Word order carries meaning: *"cancel my plan"* $\neq$ *"plan my cancellation"*, yet attention alone treats them as the same bag. We add a **positional encoding** (fixed sinusoidal or learned) to each token embedding so that position is represented.

(f) From the same encoder output, how do you produce (i) the single intent label for the message, and (ii) the per-word entity tags?

Solution:

- **Intent (one label per message)**: take the final `[CLS]` embedding and pass it through a linear + softmax head over the intents.
- **Entities (one label per token)**: apply a shared linear head to **every** token's output embedding to predict a per-token tag (B-I-O scheme). Because the embeddings are contextual, the same word can get different tags depending on context.

(g) You will start from a pretrained model rather than training from scratch. Would you choose an encoder-only (BERT-style), a decoder-only (GPT-style), or an encoder–decoder model for these two tasks? Why, and how do you use it?

Solution: Encoder-only (BERT-style). Both tasks are understanding tasks, and an encoder gives each token **bidirectional** context (left and right), which is what classification and entity tagging need.

Use **transfer learning**: keep the pretrained encoder and **fine-tune** it with the two small task heads on the labelled data – far less data and compute than training from scratch.

(h) Messages have very different lengths, but you train in batches. What must the attention handle, and how?

Solution: Pad every message in a batch to a common length, then use a **padding mask**: set the attention scores at padding positions to $-\infty$ before the softmax, so real tokens never attend to padding (and padding contributes nothing).

(i) The client now also wants the system to draft a suggested reply to each message. What architecture would you use, and how does the attention change compared to the classifier?

Solution: Use a **decoder** (or an encoder–decoder model).

- **Masked (causal) self-attention**: a position may only attend to earlier positions, so text is generated one token at a time (autoregressively).
- **Cross-attention** (encoder–decoder): the decoder attends to the encoded customer message (queries from the decoder, keys/values from the encoder), so the reply is conditioned on the incoming message.

(j) While fine-tuning the classifier you observe that the **training loss keeps decreasing but the validation loss starts rising after only one or two epochs**. What are the possible alterations you might try?

Solution: This is over-fitting during fine-tuning on limited labelled data. Options:

- Lower the learning rate; use fewer epochs / **early stopping**; add a warm-up + decay schedule.
- **Freeze** the lower Transformer layers and train mainly the head (or unfreeze gradually).
- Regularise: dropout, weight decay; use a smaller pretrained model; get more or **augmented** data (e.g. back-translation).

(k) Some transcripts are thousands of tokens long, and training runs out of memory on these. Why does this happen, and give one way to mitigate it.

Solution: Self-attention compares every token with every other token, so compute and memory grow $\mathcal{O}(n^2)$ with the sequence length n.

Mitigations: truncate or split long messages into chunks; or use an **efficient/sparse-attention** model (e.g. Longformer) or a longer-context model designed for this.

(l) (Vision aside.) The same company also wants to classify small product screenshots. How can the Transformer encoder be applied to an image?

Solution: Use a **Vision Transformer (ViT)**: split the image into fixed patches (e.g. 16×16), flatten and **linearly project** each patch into an embedding, prepend a `[CLS]` token, add positional encodings, and feed the sequence to the encoder; classify from the `[CLS]` output. Note ViT has weak built-in inductive bias, so it is data-hungry and benefits from large-scale pretraining.